LEZIONI 20, 23, 26
Approaches to Test Generation

Ingegneria del Software 2 – Software Testing

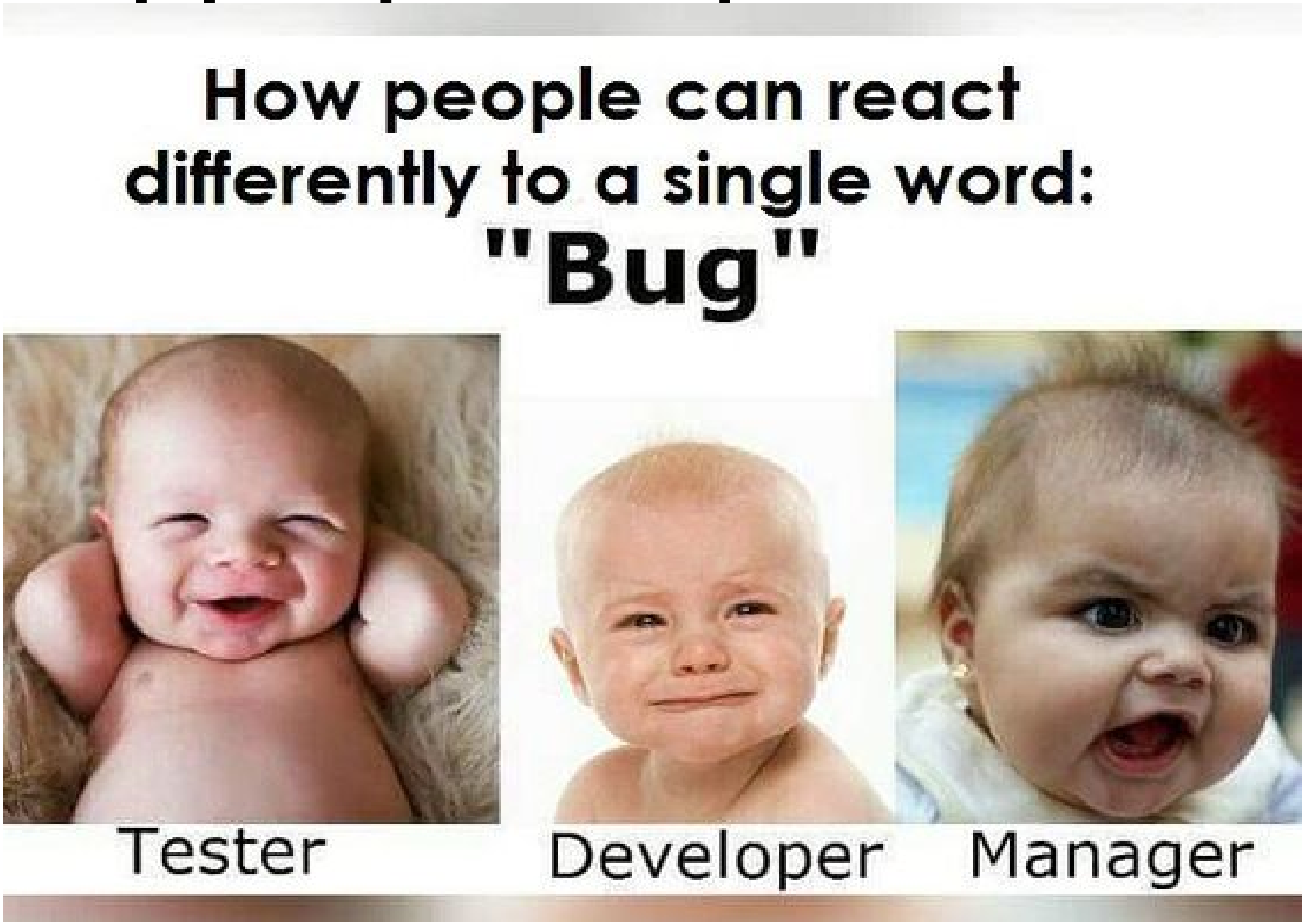
Università degli Studi di Tor Vergata - Roma

Guglielmo De Angelis
guglielmo.deangelis@iasi.cnr.it

… approccio per un’appropriata pianificazione



… approccio per un’appropriata pianificazione



software testing – le domande fondamentali

- quali/quanti input selezionare?
  - scelta di valori rappresentativi
  - differenti approcci:
    - uso di dati da esecuzioni storiche
    - selezione randomica degli input
    - generazione automatica rispetto ad un obiettivo di adequacy (vedi lezioni successive)
    - partizione del dominio in classi di equivalenza rispetto agli input attesi ed allo stato del SUT
    - interrogazione di LLM
- quando posso smettere di testare?
  - definizione dei criteri di copertura
    - statement, archi, condizioni, flussi, funzionalità, etc. ect.
  - definizione di valori minimi per l'accettazione
- come faccio a sapere se il risultato di una operazione è corretto?
  - oracle problem
  - si deducono valori attesi da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che devono valere, etc.

software testing – le domande fondamentali

- quali/quanti input selezionare?
  - scelta di valori rappresentativi
  - differenti approcci:
    - uso di dati da esecuzioni storiche
    - selezione randomica degli input
    - generazione automatica rispetto ad un obiettivo di adequacy
    - partizione del dominio in classi di equivalenza rispetto agli input attesi ed allo stato del SUT
    - interrogazione di LLM
- quando posso smettere di testare?
  - definizione dei criteri di copertura
    - statement, archi, condizioni, flussi, funzionalità, etc. ect.
  - definizione di valori minimi per l'accettazione
- come faccio a sapere se il risultato di una operazione è corretto?
  - oracle problem
  - si deducono valori attesi da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che devono valere, etc.

software testing – le domande fondamentali

- quali/quanti input selezionare?
- uso di dati da esecuzioni storiche
  - **vantaggi**:
    - modi di utilizzo effettivamente sperimentati nel sistema
    - spesso si rivela un ottimo metodo per far focalizzarsi sugli aspetti effettivamente più sollecitate
    - sicuramente utile nelle attività che guardano alla validazione del sistema
    - può dare indicazioni sulla realiability effettiva del sistema in operazione
  - **svantaggi**:
    - non risolve il problema della selezione o della prioritizzazione.
    - alto numero di casi di test dagli storici
    - non applicabile su nuove funzionalità/aspetti del sistema
- si deducono valori attesi da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che devono valere, etc.

software testing – le domande fondamentali

- quali/quanti input selezionare?
  - scelta di valori rappresentativi
  - differenti approcci:
    - uso di dati da esecuzioni storiche
    - selezione randomica degli input
    - generazione automatica rispetto ad un obiettivo di adequacy
    - partizione del dominio in classi di equivalenza rispetto agli input attesi ed allo stato del SUT
    - interrogazione di LLM
- quando posso smettere di testare?
  - definizione dei criteri di copertura
    - statement, archi, condizioni, flussi, funzionalità, etc. ect.
  - definizione di valori minimi per l'accettazione
- come faccio a sapere se il risultato di una operazione è corretto?
  - oracle problem
  - si deducono valori attesi da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che devono valere, etc.

software testing – le domande fondamentali

- quali/quanti input selezionare?
- scelta randomica dai valori di input (o dei casi di test)
  - **vantaggi**:
    - apparente semplicità nella realizzazione del metodo
    - spesso si rivela un ottimo metodo per far emergere malfunzionamenti grossolani
    - sicuramente utile nelle fasi iniziali del testing
  - **svantaggi**:
    - ricerca cieca
    - alto numero di casi di test da eseguire per avere livelli di confidenza sufficientemente significativi
    - in pratica la realizzazione di un processo effettivamente randomico potrebbe più difficoltoso del previsto
      - particolari attenzioni nella individuazione e rimozioni di eventuali bias nascosti
- si deducono valori attesi da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che devono valere, etc.

software testing – le domande fondamentali

- quali/quanti input selezionare?
  - scelta randomica dai valori di input (o dei ca
    - **vantaggi**:
      - apparente semplicità nella realizza etodo
      - spesso si rivela un ottimo metod nergere malfunzionamenti grossolani
      - sicuramente utile nelle fasi esting
    - **svantaggi**:
      - ricerca cieca
      - alto numero di casi seguire per avere livelli di confidenza suffici significativi
      - in pratica la re di un processo effettivamente randomico difficoltoso del previsto
      - particol ni nella individuazione e rimozioni di event ascosti
  - si ded da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che dev

APPROFONDIMENTO DA SLIDE 21

software testing – le domande fondamentali

- quali/quanti input selezionare?
  - scelta di valori rappresentativi
  - differenti approcci:
    - uso di dati da esecuzioni storiche
    - selezione randomica degli input
    - generazione automatica rispetto ad un obiettivo di adequacy
    - partizione del dominio in classi di equivalenza rispetto agli input attesi ed allo stato del SUT
    - interrogazione di LLM
- quando posso smettere di testare?
  - definizione dei criteri di copertura
    - statement, archi, condizioni, flussi, funzionalità, etc. ect.
  - definizione di valori minimi per l'accettazione
- come faccio a sapere se il risultato di una operazione è corretto?
  - oracle problem
  - si deducono valori attesi da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che devono valere, etc.

software testing – le domande fondamentali

- quali/quanti input selezionare?
- le misure di adequacy stimano la bontà di un insieme di test. Generare test automaticamente guidati da tali metriche
  - vantaggi:
    - in molti case l'approccio è altamente automatizzabile
    - approcci facilmente integrabili con i processi di sviluppo e di CI
  - **svantaggi**:
    - nei casi in cui la generazione è guidata dal SUT ne possono risultare molti test poco significativi per le effettive condizioni di utilizzo del sistema
    - nei casi in cui la generazione è guidata da modelli del SUT si hanno strumenti di generazione meno versatili nell'integrazione con i processi di sviluppo
    - complessità legate alla scelta degli iper-parametri ed alle euristiche di generazione
- si deducono valori attesi da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che devono valere, etc.

software testing – le domande fondamentali

- quali/quanti input selezionare?
- le misure di adequacy stimano la bontà d test.
  Generare test automaticamente guida ne
  - vantaggi:
    - in molti case l'approccio è alt tizzabile
    - approcci facilmente integra ssi di sviluppo e di CI
  - **svantaggi**:
    - nei casi in cui la gene ata dal SUT ne possono risultare molti test r per le effettive condizioni di utilizzo del siste
    - nei casi in cui è guidata da modelli del SUT si hanno strum zione meno versatili nell'integrazione con i pro o
    - compl a scelta degli iper-parametri ed alle euri azione
- si ded da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che devono v

APPROFONDIMENTO NELLE LEZIONI SUCCESSIVE

software testing – le domande fondamentali

- quali/quanti input selezionare?
  - scelta di valori rappresentativi
  - differenti approcci:
    - uso di dati da esecuzioni storiche
    - selezione randomica degli input
    - generazione automatica rispetto ad un obiettivo di adequacy
    - partizione del dominio in classi di equivalenza rispetto agli input attesi ed allo stato del SUT
    - interrogazione di LLM
- quando posso smettere di testare?
  - definizione dei criteri di copertura
    - statement, archi, condizioni, flussi, funzionalità, etc. ect.
  - definizione di valori minimi per l'accettazione
- come faccio a sapere se il risultato di una operazione è corretto?
  - oracle problem
  - si deducono valori attesi da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che devono valere, etc.

software testing – le domande fondamentali

- quali/quanti input selezionare?
  - partizionare il dominio di input (parametri e stato del SUT) e definire almeno un test per ogni partizione
  - **vantaggi**:
    - indipendente dall'implementazione
    - nella fase di progettazione dei test si cerca di identificare aspetti salienti del dominio modi di utilizzo effettivamente sperimentati nel sistema
  - **svantaggi**:
    - la ovviamente la qualità dei test è funzione dei livelli chiarezza dei requisiti e delle specifiche
    - si riescono a prende in considerazione solo vincoli o limitazioni esplicitamente espressi
-
  - familiarità del SUT
  - conoscenza dei contesti operativi e di deployment
  - si deducono valori attesi da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che devono valere, etc.

software testing – le domande fondamentali

- quali/quanti input selezionare?
- partizionare il dominio di input (parametri e UT) e definire almeno un test per ogni partizione
- **vantaggi**:
  - indipendente dall'implementazio
  - nella fase di progettazione dei ca di identificare aspetti salienti del dominio modi di ettivamente sperimentati nel sistema
- **svantaggi**:
  - la ovviamente la qu t è funzione dei livelli chiarezza dei requisiti e dell e
  - si riescono a p nsiderazione solo vincoli o limitazioni esplicitamen
  - familiarità
  - conosc ntesti operativi e di deployment
- si ded da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che dev

APPROFONDIMENTO DA SLIDE 28

software testing – le domande fondamentali

- quali/quanti input selezionare?
  - scelta di valori rappresentativi
  - differenti approcci:
    - uso di dati da esecuzioni storiche
    - selezione randomica degli input
    - generazione automatica rispetto ad un obiettivo di adequacy
    - partizione del dominio in classi di equivalenza rispetto agli input attesi ed allo stato del SUT
    - interrogazione di LLM
- quando posso smettere di testare?
  - definizione dei criteri di copertura
    - statement, archi, condizioni, flussi, funzionalità, etc. ect.
  - definizione di valori minimi per l'accettazione
- come faccio a sapere se il risultato di una operazione è corretto?
  - oracle problem
  - si deducono valori attesi da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che devono valere, etc.

software testing – le domande fondamentali

- quali/quanti input selezionare?
- uso di forme di AI altamente addestrate, interazioni specifcate per mezzo del linguaggio naturale
  - **vantaggi**:
    - <u>apparente</u> semplicità di interazione con LLM
    - grande potenza di ragionamento in tempi relativamente ristretti
    - <u>apparente</u> onniscenza degli LLM
  - **svantaggi**:
    - soluzioni funzione della formulazione del problema (prompt engineering)
    - eccessivo trust nel LLM (le risposte osservate potrebbero essere frutto di bias accidentali)
    - ragionamento black-box e probabilistico (gli outcome non possono essere considerati deterministici)
- si deducono valori attesi da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che devono valere, etc.

software testing – le domande fondamentali

- quali/quanti input selezionare?
- uso di forme di AI altamente addestrate, int ecifcate per mezzo del linguaggio naturale
  - **vantaggi**:
    - apparente semplicità di interazio
    - grande potenza di ragionamen relativamente ristretti
    - apparente onniscenza degl
  - **svantaggi**:
    - soluzioni funzione dell one del problema (prompt engineering)
    - eccessivo trust n isposte osservate potrebbero essere frutto di bias ac
    - ragionamen x e probabilistico (gli outcome non possono siderati deterministici)
- si ded da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che dev

APPROFONDIMENTO DA SLIDE 53

software testing – le domande fondamentali

- quali/quanti input selezionare?
  - scelta di valori rappresentativi
  - differenti approcci:
    - uso di dati da esecuzioni storiche
    - selezione randomica degli input
    - generazione automatica rispetto ad un obiettivo di adequacy
    - partizione del dominio in classi di equivalenza rispetto agli input attesi ed allo stato del SUT
    - interrogazione di LLM
- quando posso smettere di testare?
  - definizione dei criteri di copertura
    - statement, archi, condizioni, flussi, funzionalità, etc. ect.
  - definizione di valori minimi per l'accettazione
- come faccio a sapere se il risultato di una operazione è corretto?
  - oracle problem
  - si deducono valori attesi da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che devono valere, etc.

software testing – le domande fondamentali

- quali/quanti input devo usare?

<u>ATTENZIONE:</u> la specifica completa di un caso di test deve prevedere l'output atteso a seguito dell'interazione con il sistema/elemento testato

Inoltre, rimarchiamo che sono plausibili casi di test che si aspettino come outcome il fallimento del SUT (per esempio attraverso il lancio di una eccezione o un errore)

- quando posso smettere di testare?
  - definizione dei criteri di copertura
    - statement, archi, condizioni, flussi, funzionalità, etc. ect.
  - definizione di valori minimi per l'accettazione
- come faccio a sapere se il risultato di una operazione è corretto?
  - oracle problem
  - si deducono valori attesi da specifiche, storici, sistemi equivalenti in esecuzione, inferenze su proprietà che devono valere, etc.

scelta randomica degli input (o dei casi di test)

scelta randomica degli input (o dei casi di test)

- Randoop è un generatore di (unit) test per programmi Java.



https://randoop.github.io/randoop/

- Crea automaticamente programmi Junit che implementano dei casi di test
- Lettura Consigliata:
  - **Feedback-directed random test generation** by Carlos Pacheco, Shuvendu K. Lahiri, Michael D. Ernst, and Thomas Ball. In ICSE '07: Proceedings of the 29th International Conference on Software Engineering, (Minneapolis, MN, USA), 2007
  - https://homes.cs.washington.edu/%7Emernst/pubs/feedback-testgen-icse2007.pdf

Randoop – esempi di test randomici

1. Useful test

```
Set s = new HashSet();
s.add("hi");
assertTrue(s.equals(s));
```

2. Redundant test

```
Set s = new HashSet();
s.add("hi");
s.isEmpty();
assertTrue(s.equals(s));
```

3. Useful test

```
Date d = new Date(2006, 2, 14);
assertTrue(d.equals(d));
```

4. Illegal test

```
Date d = new Date(2006, 2, 14);
d.setMonth(-1); // pre: argument >= 0
assertTrue(d.equals(d));
```

5. Illegal test

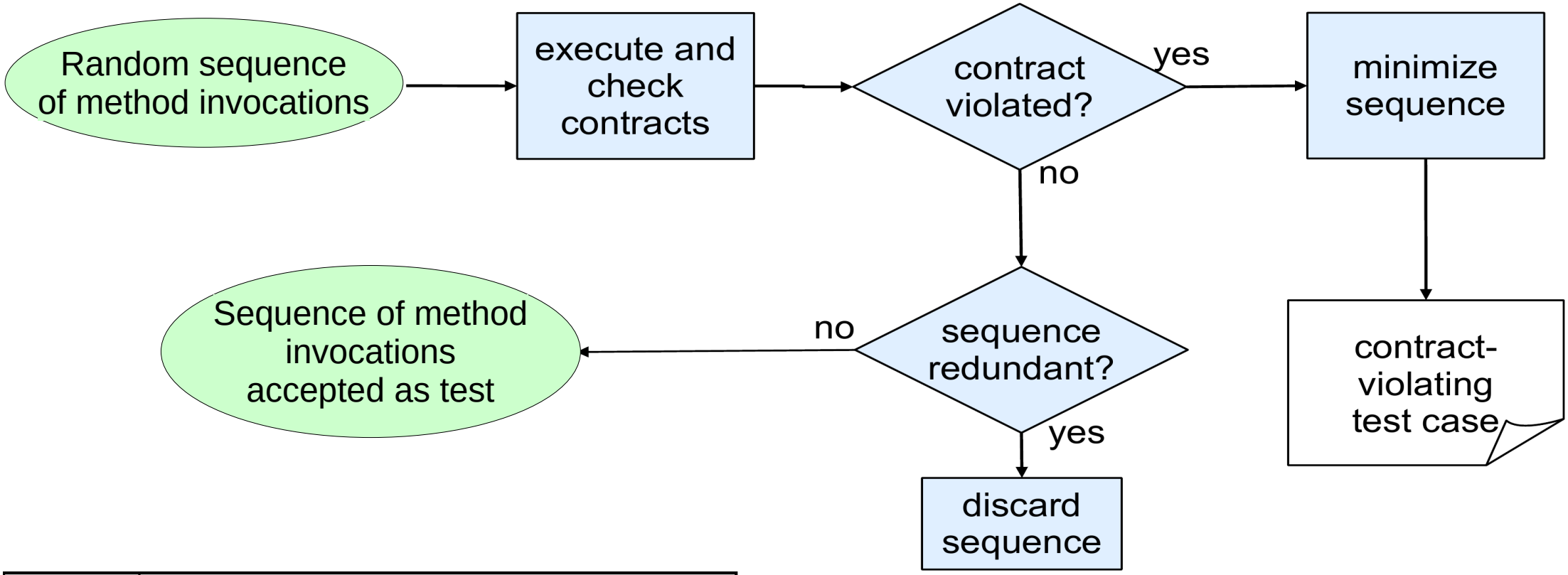
```
Date d = new Date(2006, 2, 14);
d.setMonth(-1);
d.setDay(5);
assertTrue(d.equals(d));
```

do not output

do not even create

Estratto dalla presentazione di "Feedback-directed random test generation" by Carlos Pacheco, Shuvendu K. Lahiri, Michael D. Ernst, and Thomas Ball ad ICSE 2007.

Randoop – singola iterazione



Method	
contract	**description**
Exception (Java)	method throws no `NullPointerException` if no input parameter was null
	method throws no `AssertionError`
Exception (.NET)	method throws no `NullReferenceException` if no input parameter was null
	method throws no `IndexOutOfRangeException`
	method throws no `AssertionError`

Contratti di default in Randoop

Object	
contract	**description**
equals	`o.equals(o)` returns `true`
	`o.equals(o)` throws no exception
hashCode	`o.hashCode()` throws no exception
toString	`o.toString()` throws no exception

Estratto dalla presentazione di "Feedback-directed random test generation" by Carlos Pacheco, Shuvendu K. Lahiri, Michael D. Ernst, and Thomas Ball ad ICSE 2007.

Randoop – how to

Installing Randoop

Randoop runs on a Java 8 or later JVM.

Download and unzip the file randoop-4.3.3.zip. This manual uses ${RANDOOP_PATH} to refer to the path of the unzipped archive, and ${RANDOOP_JAR} to refer to the location of randoop-all-4.3.3.jar within the unzipped archive.

Alternately, if you want to view or work with the source code, follow the instructions in the Getting Started section of the Randoop Developer's Manual.

You may wish to join the randoop-discuss@googlegroups.com mailing list so that you are notified of new releases.

Running Randoop

Run Randoop by invoking its main class randoop.main.Main:

```
java randoop.main.Main command args...
```

Randoop supports three commands:

- **gentests** generates unit tests. For details, see Generating Tests. Example use:

  ```
  java -Xmx3000m -classpath myclasspath:${RANDOOP_JAR} randoop.main.Main gentests --testclass=java.util.TreeSet --output-limit=100
  ```

  (However, note that it is extremely unusual to use the --testclass command-line argument to specify just one class under test.)
- **minimize** minimizes a failing JUnit test suite. For details, see Minimizing a failing JUnit test suite. Example use:

  ```
  java -cp ${RANDOOP_JAR} randoop.main.Main minimize --suitepath=ErrorTest0.java --suiteclasspath=myclasspath
  ```

- **help** prints out a usage message. For details, see Getting Help. Example uses:

  ```
  java -classpath ${RANDOOP_JAR} randoop.main.Main help
  java -classpath ${RANDOOP_JAR} randoop.main.Main help gentests
  java -classpath ${RANDOOP_JAR} randoop.main.Main help minimize
  ```

(On Windows, adjust the classpath, such as using semicolon instead of colon as the separator.)

https://randoop.github.io/randoop/manualindex.html#running_randoop

Randoop – how to

- Randoop generarà test per un metodo o un costruttore M solo se tutte le seguenti condizioni sono soddisfatte
  - M è riferibile attraverso uno dei seguenti parametri:
    - --testjar
    - --classlist
    - --testclass
    - --methodlist.
  - M non è stato esplicitamente escluso dalla generazione
    - --omit-methods
    - --omit-methods-file
  - la classe di M, il package che la contiene ed il tipo ritornato da M sono inclusi nel classpath di Randoop
    - ad esempio attraverso il parametro: --junit-package-name

Randoop – indicazioni operative

- (se volete) potete lavorare offline con Randoop senza integrarlo nel processo di build MVN
- generate i test localmente ed includete nel repo solo quelli che vi sono utili al fine dello svolgimento della sperimentazione

classi di equivalenza … come?

classi di equivalenza … come?

- partizionare il dominio di input
  - per ogni elemento in una partizione il SUT deve evidenziare lo stesso comportamento … o dalle specifiche si può assumere che cosi sia!!
- includere nella test suite in costruzione (i.e., T_g) un elemento per ogni partizione
- ovviamente la qualità di T_g sarà funzione dei livelli
  - chiarezza dei requisiti e delle specifiche
    - vincoli o limitazioni esplicitamente espressi
  - familiarità del SUT
    - conoscenza dei contesti operativi e di deployment
  - conoscenza dell'implementazione
    - dettagli su come sono realizzate procedure e controlli

classi di equivalenza – linee guida generali

- considerare uno o più parametri di input per l'attività di test
  - direttamente derivante dal dominio di sistema
    - key abstraction, feature, needs, requirements
  - esplicitato al solo livello di astrazione considerato
    - funzione di una particolare scelta implementativa
- partizionare lo spazio dei valori ammissibili basandosi su conoscenza esplicita o implicita di:
  - range (impliciti o espliciti): considerare un valore nel range e due valori al di fuori
  - stringhe: considerare almeno un insieme di stringhe tutte valide, un insieme di stringhe tutte non valide, un insieme con stringhe vuote ed un insieme con stringhe nulle
  - enumerazioni: considerare ogni valore assegnato ad una classe di equivalenza differente
  - array: considerare almeno una partizione con array tutti legali, una con array tutti vuoti, ed una dove tutti gli array superano la lunghezza massima consentita
  - tipi di dato complesso: considerare le partizioni applicando iterativamente i criteri generali ai dati nella struttura. Ricordarsi di considerare i valori nulli!

classi di equivalenza – linee guida generali

- considerare uno o più parametri di input per l'attività di test
  - direttamente derivante dal dominio di sistema
    - key abstraction, feature, needs, requirements
  - esplicitato al solo livello di astrazione considerato
    - funzione di una particolare scelta implementativa

ATTENZIONE: queste sono linee guida generali principalmente basate sul tipo sintattico dei parametri osservati. E' giusto considerarle ma non in modo esclusivo.
Spesso classi di equivalenza significative possono essere dedotte dalla semantica del parametro/variabile/attributo osservato:

`String password` ⇒
- ""
- `null`
- password valida nel dominio considerato
  - password corretta
  - password errata
- password non valida nel dominio considerato

classi di equivalenza – linee guida generali

- considerare uno o più parametri di input per l'attività di test
  - direttamente derivante dal dominio di sistema
    - key abstraction, feature, needs, requirements

ATTENZIONE: non confondete il concetto di VALIDITA' di un valore assegnabile ad un parametro con il concetto di CORRETTEZZA legato ad una specifica configurazione del SUT

Dato il SUT: `DAOUser.loadUser`, e considerato:

```
User u = DAOUser.getInstance().loadUser(email)
```

si possono valutare i seguenti scenari:

- `email="-11jud7@sjlkla9873@jjdud." // NON VALIDA`
- `email="mrfoo@nothing.org" // VALIDA, ma potrebbe essere "CORRETTA" o "NON CORRETTA" dipendentemente dall'ESISTENZA dell'utente nella configurazione del SUT`

criteri per identificare i test

- unidimensionale
  - ogni parametro di input al test è considerato in modo indipendente
  - i test sono scelti/sintetizzati per coprire tutte le classi di equivalenza considerate
- multidimensionale
  - viene considerato il prodotto cartesiano dei parametri di input
  - i test sono scelti/sintetizzati coprendo la matrice di combinazione della classi di equivalenza

criteri per identificare i test

- unidimensionale
  - ogni parametro di input al test è considerato in modo indipendente
  - i test sono scelti/sintetizzati per coprire tutte le classi di equivalenza considerate
- multidimensionale
  - viene considerato il prodotto cartesiano dei parametri di input
  - i test sono scelti/sintetizzati coprendo la matrice di combinazione della classi di equivalenza

procedura sistematica per il partizionamento

1. identificare i domini di input
   - dalla definizione di key abstraction, feature, needs, requisiti
   - dai tipi di dato legati particolare scelte implementative
   - da condizioni imposte (esplicitamente o implicitamente)
2. identificare le classi di equivalenza per ogni parametro del test
3. combinare le classi di equivalenza
4. eliminare combinazioni di classi di equivalenza non ammissibili

torniamo all'esempio

- considerare la seguente operazione Java
  - dalla classe `LedgerHandle` di Apache Bookkeeper, link: https://github.com/apache/bookkeeper/blob/e7430ce25e3609bf77b028e4f0475dad2147c22d/bookkeeper-server/src/main/java/org/apache/bookkeeper/client/LedgerHandle.java#L797

```
void asyncReadEntriesInternal(long firstEntry,
long lastEntry, ReadCallback cb, Object ctx,
boolean isRecoveryRead)
```

ATTENZIONE: solo per semplicità stiamo considerando che
- il SUT sia un singolo metodo. In generale non è questo il caso, e per esempio viene considerato come SUT una intera classe, un intero package, o un modulo (rappresentato da un insieme di classi)
- i parametri osservati nel SUT corrispondono ai parametri formali del metodo. In generale non è questo il caso ma vengono presi in considerazioni anche lo stato del SUT (e.g., insieme dei valori negli attributi), i valori presenti nello strato di persistenza, lo stato di altre istanze nel sistema

torniamo all’esempio

- considerare la seguente operazione Java
  - dalla classe `LedgerHandle` di Apache Bookkeeper, link: https://github.com/apache/bookkeeper/blob/e7430ce25e3609bf77b028e4f0475dad2147c22d/bookkeeper-server/src/main/java/org/apache/bookkeeper/client/LedgerHandle.java#L797

```
void asyncReadEntriesInternal(long firstEntry,
long lastEntry, ReadCallback cb, Object ctx,
boolean isRecoveryRead)
```

torniamo all’esempio

- considerare la seguente operazione Java
  - dalla classe `LedgerHandle` di Apache Bookkeeper, link: https://github.com/apache/bookkeeper/blob/e7430ce25e3609bf77b028e4f0475dad2147c22d/bookkeeper-server/src/main/java/org/apache/bookkeeper/client/LedgerHandle.java#L797

```
void asyncReadEntriesInternal(long firstEntry, long lastEntry, ReadCallback cb, Object ctx, boolean isRecoveryRead)
```

- classi di equivalenza
  - `isRecoveryRead:` {false}, {true}
  - `cb:` {null}, {“valid_instance”}, {“invalid_instance”}
  - `ctx`: {null}, {“valid_instance”}, {“invalid_instance”}
  - `firstEntry`: {≤0}, {>0}
  - `lastEntry`: {≤0}, {>0}
  - `LedgerHandle:` {vuoto}, {ha sufficienti entry}, {non ha sufficienti entry}

torniamo all'esempio

ATTENZIONE: sui parametri numerici, non basarsi unicamente sulla classificazione: "positivo" "negativo"

Come nel precedente esempio della password come stringa va/ valutare classi di equivalenza in base alla semantica del parametro ed al contesto considerato.

In questo specifico caso "`firstEntry`" e "`lastEntry`" rappresentano indici, quindi …

vo
`ct`

- `ctx`: {null}, {"valid_instance"}, {"invalid_instance"}
- `firstEntry`: {≤0}, {>0}
- `lastEntry`: {≤0}, {>0}
- `LedgerHandle:` {vuoto}, {ha sufficienti entry}, {non ha sufficienti entry}

torniamo all’esempio

riusciamo a fare qualcosa in più ?!?!?

possiamo sfruttare qualche informazione
(in questo caso implicita)
per meglio identificare classi di equivalenza ?

- c/main/java/

```
void
ctx,
```

```
Object
```

- isRecoveryRead: {false}, {true}
  - cb: {null}, {“valid_instance”}, {“invalid_instance”}
  - ctx: {null}, {“valid_instance”}, {“invalid_instance”}
  - firstEntry: {≤0}, {>0}
  - lastEntry: {≤0}, {>0}
  - LedgerHandle: {vuoto}, {ha sufficienti entry}, {non ha sufficienti entry}

torniamo all’esempio

- considerare la seguente operazione Java
  - dalla classe `LedgerHandle` di Apache Bookkeeper, link: https://github.com/apache/bookkeeper/blob/e7430ce25e3609bf77b028e4f0475dad2147c22d/bookkeeper-server/src/main/java/org/apache/bookkeeper/client/LedgerHandle.java#L797

```
void asyncReadEntriesInternal(long firstEntry, long lastEntry, ReadCallback cb, Object ctx, boolean isRecoveryRead)
```

- classi di equivalenza
  - `isRecoveryRead:` {false}, {true}
  - `cb:` {null}, {“valid_instance”}, {“invalid_instance”}
  - `ctx`: {null}, {“valid_instance”}, {“invalid_instance”}
  - `firstEntry`: {≤0}, {>0}
  - `lastEntry`: {<`firstEntry`}, {=`firstEntry`}, {>`firstEntry`}
  - `LedgerHandle:` {vuoto}, {ha sufficienti entry}, {non ha sufficienti entry}

torniamo all’esempio

- c/main/java/org/

```
void
bool                                                    Object ctx,
```

- isRecoveryRead: {false}, {true}
  - cb: {null}, {“valid_instance”}, {“invalid_instance”}
  - ctx: {null}, {“valid_instance”}, {“invalid_instance”}
  - firstEntry: {≤0}, {>0}
  - lastEntry: {<firstEntry}, {=firstEntry}, {>firstEntry}
  - LedgerHandle: {vuoto}, {ha sufficienti entry}, {non ha sufficienti entry}

lascio a voi l’esercizio di pensare a come migliorare le classi di equivalenza considerando anche `LedgerHandle`

torniamo all'esempio

ATTENZIONE: escludere i casi di input "`invalid_instance`" per tipi di dato complesso giustificando la scelta con considerazioni tipo:

- "IL COSTRUTTORE DELLA CLASSE TORNA SOLO ISTANZE CORRETTE"

non è accettabile. Infatti:

- il codice del costruttore potrebbe cambiare introducendo bug
- soluzioni polimorfe potrebbero consentire di legare istanze di sottoclassi che fanno riferimento ad implementazioni del costruttore diverse da quelle considerate

- `isRecoveryRead`: {false}, {true}
- `cb`: {null}, {"valid_instance"}, {"invalid_instance"}
- `ctx`: {null}, {"valid_instance"}, {"invalid_instance"}
- `firstEntry`: {≤0}, {>0}
- `lastEntry`: {<`firstEntry`}, {=`firstEntry`}, {>`firstEntry`}
- `LedgerHandle`: {vuoto}, {ha sufficienti entry}, {non ha sufficienti entry}

analisi dei boundary-value

- l'esperienza … comprovata anche da studi empirici, suggerisce che i più comuni bug di programmazione sono introdotti su valori “a confine” delle classi di equivalenza
- selezionare valori rappresentativi per una classe di equivalenza, tale che siano al confine della classe stessa
- schema tipo per selezionare test
  1. partizionare il dominio di riferimento per ognuno dei parametri osservati/considerati
  2. identificare i confini di ogni partizione
  3. selezionare i valori in modo tale che ogni confine occorra in almeno un test

analisi dei boundary-value

- l'esperienza … comprovata anche da studi empirici, suggerisce che i più comuni bug di programmazione sono introdotti su valori “a confine” delle classi di equivalenza
- selezionare valori rappresentativi per una classe di equivalenza, tale che siano al confine della classe stessa
- schema tipo per selezionare tuple di input
  1. partizionare il dominio di riferimento per ognuno dei parametri osservati/considerati
  2. identificare i confini di ogni partizione
  3. selezionare i valori in modo tale che ogni confine occorra
     - in tutte le possibili combinazioni degli altri (caso multidimensionale)
     - in almeno una tupla di input (caso unidimensionale)

ancora sull’esempio

- considerare la seguente operazione Java
  - dalla classe `LedgerHandle` di Apache Bookkeeper, link: https://github.com/apache/bookkeeper/blob/e7430ce25e3609bf77b028e4f0475dad2147c22d/bookkeeper-server/src/main/java/org/apache/bookkeeper/client/LedgerHandle.java#L797

```
void asyncReadEntriesInternal(long firstEntry, long lastEntry,
ReadCallback cb, Object ctx, boolean isRecoveryRead)
```

- limitiamoci a considerare le classi di equivalenza per i soli `firstEntry` e `lastEntry`
  - gli altri parametri li assumiamo momentaneamente fissati
- boundary-values

ancora sull’esempio

- considerare la seguente operazione Java
  - dalla classe `LedgerHandle` di Apache Bookkeeper, link: https://github.com/apache/bookkeeper/blob/e7430ce25e3609bf77b028e4f0475dad2147c22d/bookkeeper-server/src/main/java/org/apache/bookkeeper/client/LedgerHandle.java#L797

```
void asyncReadEntriesInternal(long firstEntry, long lastEntry,
ReadCallback cb, Object ctx, boolean isRecoveryRead)
```

- limitiamoci a considerare le classi di equivalenza per i soli `firstEntry` e `lastEntry`
  - gli altri parametri li assumiamo momentaneamente fissati
- boundary-values
  - `firstEntry`: -1; 0; 1;
  - `lastEntry`: `firstEntry-1`; `firstEntry`; `firstEntry+1`

ancora sull'esempio

- considerare la seguente operazione Java
  - dalla classe `LedgerHandle` di Apache Bookkeeper, link: https://github.com/apache/bookkeeper/blob/e7430ce25e3609bf77b028e4f0475dad2147c22d/bookkeeper-server/src/main/java/org/apache/bookkeeper/client/LedgerHandle.java#L797

```
void asyncReadEntriesInternal(long firstEntry, long lastEntry,
ReadCallback cb, Object ctx, boolean isRecoveryRead)
```

- limitiamoci a considerare le classi di equivalenza per i soli `firstEntry` e `lastEntry`
  - gli altri parametri li assumiamo momentaneamente fissati
- boundary-values
  - `firstEntry`: -1; 0; 1;
  - `lastEntry`: `firstEntry-1`; `firstEntry`; `firstEntry+1`

- `firstEntry`: -1 ; `lastEntry`: -2
- `firstEntry`: 0 ; `lastEntry`: -1
- `firstEntry`: 1 ; `lastEntry`: 0
- `firstEntry`: -1 ; `lastEntry`: -1
- `firstEntry`: 0 ; `lastEntry`: 0
- `firstEntry`: 1 ; `lastEntry`: 1
- `firstEntry`: -1 ; `lastEntry`: 0
- `firstEntry`: 0 ; `lastEntry`: 1
- `firstEntry`: 1 ; `lastEntry`: 2

ancora sull'esempio

- considerare la seguente operazione Java
  - dalla classe `LedgerHandle` di Apache Bookkeeper, link: https://github.com/apache/bookkeeper/blob/e7430ce25e3609bf77b028e4f0475dad2147c22d/bookkeeper-server/src/main/java/org/apache/bookkeeper/client/LedgerHandle.java#L797

```
void asyncReadEntriesInternal(long firstEntry, long lastEntry,
ReadCallback cb, Object ctx, boolean isRecoveryRead)
```

- limitiamoci a considerare le classi di equivalenza per i soli `firstEntry` e `lastEntry`
  - gli altri parametri li assumiamo momentaneamente fissati
- boundary-values
  - `firstEntry`: -1; 0; 1;
  - `lastEntry`: `firstEntry-1`; `firstEntry`; `firstEntry+1`
- possibile selezione unidimensionale secondo boundary-values
  (i.e., ogni confine appare in almeno una tupla di input, slide precedenti)

???????

ancora sull'esempio

- considerare la seguente operazione Java
  - dalla classe `LedgerHandle` di Apache Bookkeeper, link: https://github.com/apache/bookkeeper/blob/e7430ce25e3609bf77b028e4f0475dad2147c22d/bookkeeper-server/src/main/java/org/apache/bookkeeper/client/LedgerHandle.java#L797

```
void asyncReadEntriesInternal(long firstEntry, long lastEntry,
ReadCallback cb, Object ctx, boolean isRecoveryRead)
```

- limitiamoci a considerare le classi di equivalenza per i soli `firstEntry` e `lastEntry`
  - gli altri parametri li assumiamo momentaneamente fissati
- boundary-values
  - `firstEntry`: -1; 0; 1;
  - `lastEntry`: `firstEntry-1`; `firstEntry`; `firstEntry+1`
- possibile selezione unidimensionale secondo boundary-values
  (i.e., ogni confine appare in almeno una tupla di input, slide precedenti)
  - `firstEntry`: -1 ; `lastEntry`: -2
  - `firstEntry`: 0 ; `lastEntry`: 0
  - `firstEntry`: 1 ; `lastEntry`: 2

ancora sull’esempio

ATTENZIONE: una selezione unidimensionale degli input per i test (e.g. secondo baundary-values) potrebbe limitare la capacità esposizione di failure

di fatto non abbiamo nessun motivo per escludere a priori le combinazioni:
`firstEntry: 1 ; lastEntry: 0` (baundary-values multidimensionale)
`firstEntry: 1 ; lastEntry: -1` (arbitraria)
le quali potrebbero dimostrarsi casi di test significativi

- `firstEntry`: -1; 0; 1;
- `lastEntry`: `firstEntry-1`; `firstEntry`; `firstEntry+1`

- possibile selezione unidimensionale secondo boundary-values
  (i.e., ogni confine appare in almeno una tupla di input, slide precedenti)
  - `firstEntry`: -1 ; `lastEntry`: -2
  - `firstEntry`: 0 ; `lastEntry`: 0
  - `firstEntry`: 1 ; `lastEntry`: 2

procedura sistematica per il partizionamento

1. identificare i domini di input
   - dalla definizione di key abstraction, feature, needs, requisiti
   - dai tipi di dato legati particolare scelte implementative
   - da condizioni imposte (esplicitamente o implicitamente)
2. identificare le classi di equivalenza per ogni parametro del test
3. combinare le classi di equivalenza
4. eliminare combinazioni di classi di equivalenza non ammissibili

category partition è un **metodo** che può essere combinato con **tecniche** di:

- manual software testing
  - approccio iniziale per capire il contesto ed il metodo
  - lavoro richiesto da questo corso
- automatic software testing
  - approccio avanzato che prevede la progettazione e lo sviluppo di strumenti (basati su euristiche o soluzioni AI) per la generazione automatica di test
  - lavoro da svolgersi in ambito di tesi … a meno che non ci siano studenti volenterosi

Large Language Models

- LLM sono implementazioni di grandi reti neurali (costruiti secondo specifiche architetture di riferimento) che sono state addestrate su grandissime moli di dati utilizzando approcci supervisionati
- uno dei loro punti di forza è l'uso di linguaggi naturali nell'interazione uomo-macchina
  - i linguaggi di programmazione sono in genere linguaggi liberi dal contesto e possono essere considerati un sotto-insieme ristretto dell'insieme dei linguaggi naturali
  - sfruttare le capacità degli LLM di elaborare informazioni da/per specifici linguaggi di programmazione
- in SE, sfruttare le capacità di elaborazione degli LLM per supportare aspetti di generazione automatica di artefatti software
  - in software testing, tra le altre cose, generare automaticamente test program

Prompting

- una interazione con un LLM avviene attraverso una richiesta in linguaggio naturale
  - normalmente riferita come: `prompt`
- è possibile definire una specifica struttura per i prompt da sottoporre ad un LLM
  - la strutturazione è abbastanza arbitraria: può riguardare aspetti sintattici, semantici, conversazionali o di contesto
  - è possibile prevedere anche delle specifiche strutture per le risposte attese da un LLM
- nell'interazione può essere prevista una fase di interazione basate su "informazioni mirate" (Retrieval-Augmented Generation - RAG) sulla base delle quali il LLM viene istruito
  - sulla struttura dei prompt in ingresso e in risposta
  - sul contesto di riferimento

pure-prompting schema

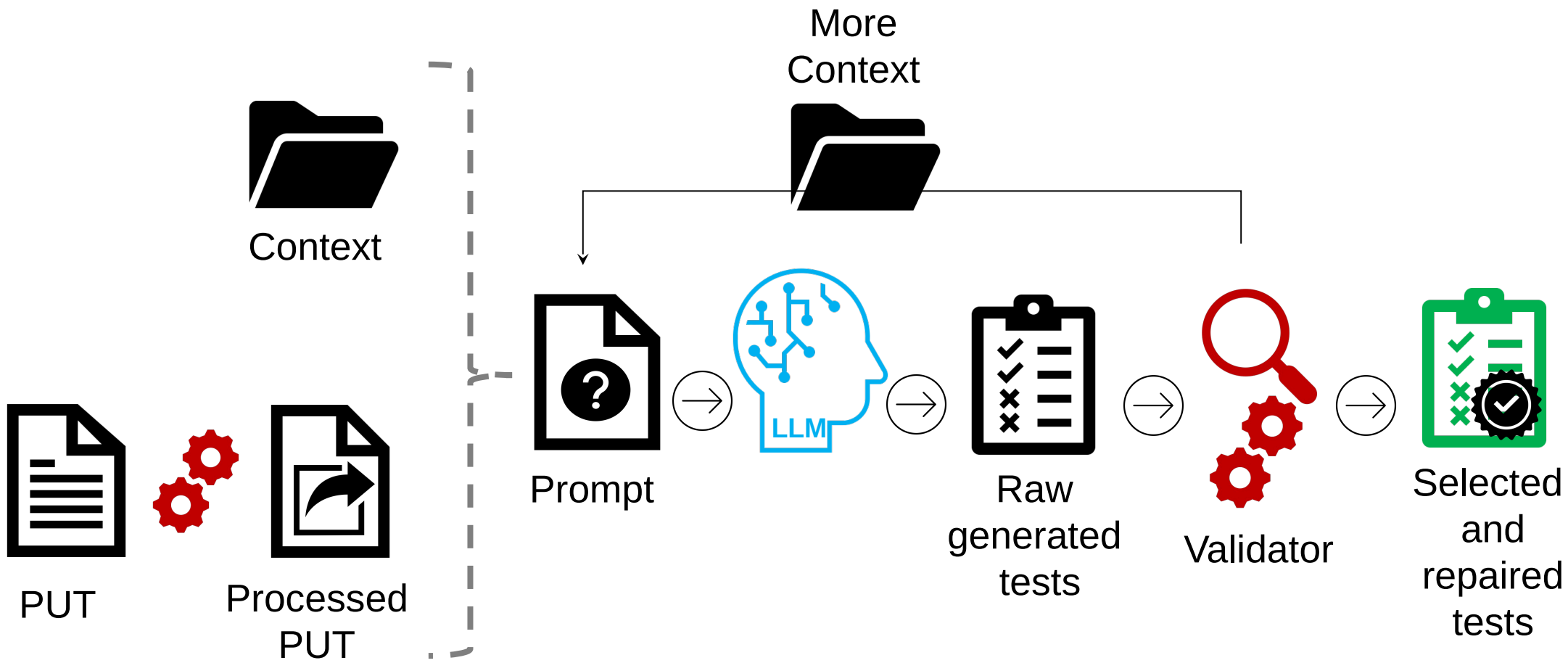


Image by Antonia Bertolino – Keynote at ICST 2025

LLM – zero shot prompting

(a) Zero-shot Prompting

```
As a professional software tester who writes Java test methods, generate a complete Junit 4
    test file ({class_name}Test.java) to comprehensively test all methods in the following
    class named {class_name}. Your output file must start with ###Test START## and finish
    with ###Test END##. Here is the source code:\n{source_code}
```

LLM – zero shot prompting

(a) Zero-shot Prompting

```
As a professional software tester who writes Java test methods, generate a complete Junit 4
    test file ({class_name}Test.java) to comprehensively test all methods in the following
    class named {class_name}. Your output file must start with ###Test START## and finish
    with ###Test END##.
```



Una variante di prompt che potrebbe rivelarsi interessante da studiare riguarda la rimozione del codice del SUT. Ad esempio:

- non esplicitando questa informazione, come verrebbe influenzato l'output?
- in base ai risultati ottenuti, potremmo intuire qualcosa sul LLM utilizzato?

LLM – few shot prompting

(b) Few-shot Prompting

```
As a professional software tester who writes Java test methods, consider the following
    examples:{fewshot_example}\nNow, generate JUnit 4 test cases to comprehensively test
    all methods in the following class named {class_name}. Your output file must start with
    ###Test START## and finish with ###Test END##. Here is the source code:\n{source_code
    }.
```

pure-prompting schema with CoT

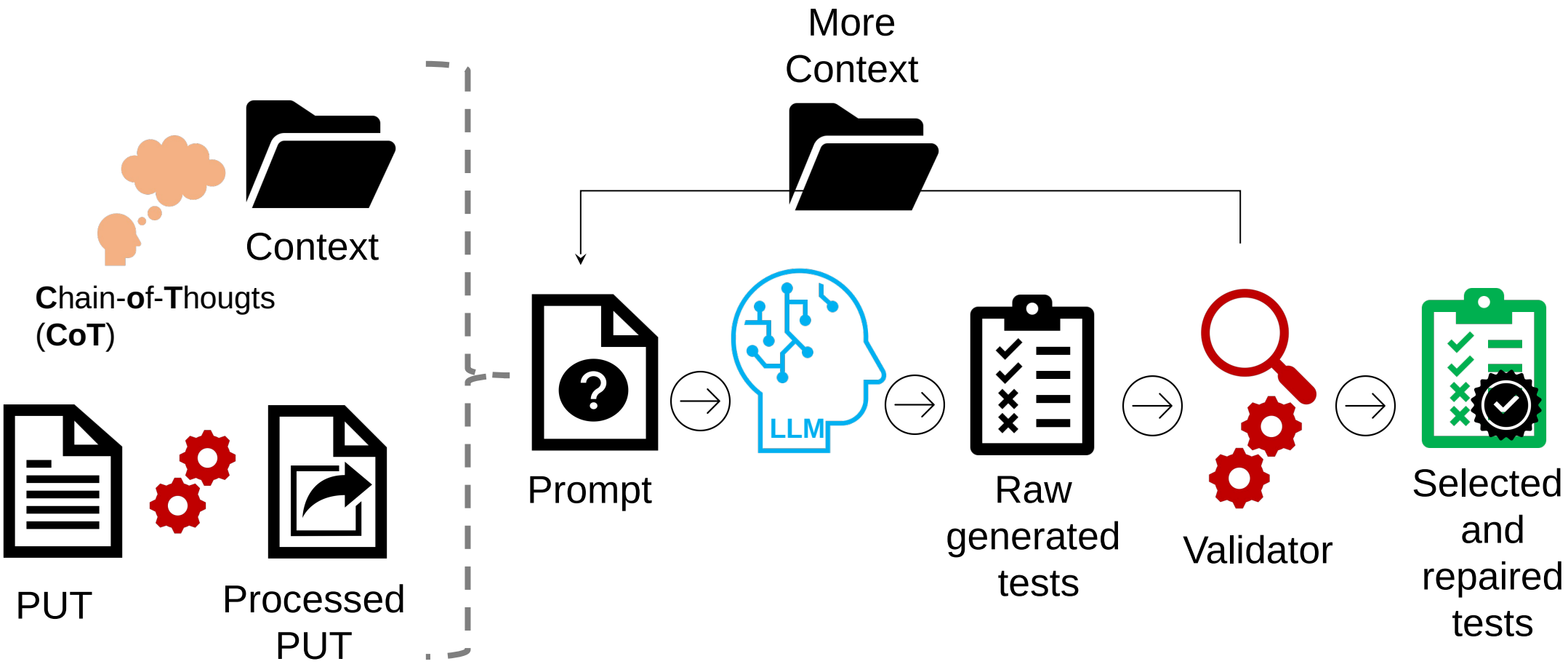


Image by Antonia Bertolino – Keynote at ICST 2025

LLM – guided tree-of-thougts prompting

(c) Guided Tree-of-Thougts Prompting

```
Imagine three different experts in software testing who are tasked with developing
    comprehensive JUnit 4 test cases for the following Java class.To comprehensively test
    all methods in the following class named {class_name} they must following these steps:\
    n- Extract and list all the public methods including their signatures\n- For each
    methods, generate a basic JUnit 4 test case that checks the method's functionality\n-
    Given the source code of the class and the listed methods, identify potential edge
    cases and exception handling scenarios that should be tested\n- Generate JUnit 4 test
    cases that specifically test for the identified edge cases and exceptions\n- Merge all
    the individual test cases into a complete JUnit 4 test file ({class_name}Test.java) for
     the given Java class.All experts will propose one test cases for each method, share it
     with the group, and then proceed to the next step. If any expert realizes they're
    wrong at any point, they leave.\nThe Java class is:\n{source_code}\nAt the end they
    must propose one complete (including typical use cases, edge cases, and error scenarios
    ) JUnit 4 test file ({class_name}Test.java) for the given Java class. The complete
    JUnit test file must start with ###Test START## and finish with ###Test END##
```

NOTA: Guided ToT può includere esempi come nel caso di few shot prompting. In questo specifico esempio non sono stati riportati

pure-prompting schema with RAGs

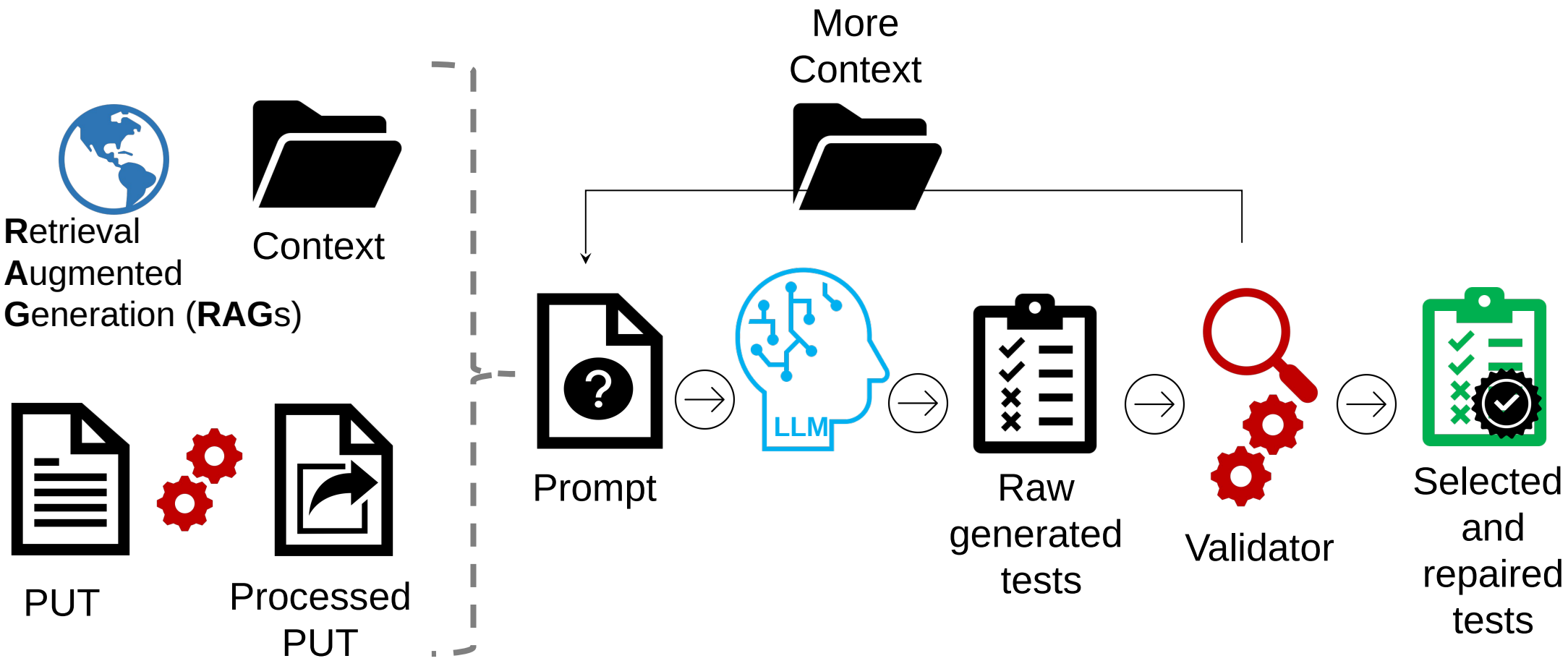


Image by Antonia Bertolino – Keynote at ICST 2025

… nel corso vi sono richiesti solo pochissimi test

- Google (Memon et al. “Taming Google-scale Continuous Testing.” SEIP@ICSE, 2017)
  - “Every day at Google 800K builds and 150M test runs are performed on more than 13K code projects”
- Bookeeper (Aprile 2023)
  - classi di test (unità): 617
  - metodi di test (@Test): 3228
- Groucho (Aprile 2023)
  - classi di test (unità): 139
  - metodi di test (@Test): 637

… nel corso vi sono richiesti solo pochissimi test

- Google (Memon et al. “Taming Google-scale Continuous Testing.” SEIP@ICSE, 2017)
  - “Every day at Google 800K builds and 150M test runs are performed on more than 13K code projects”
- Bookeeper (Aprile 2023)
  - classi di test (unità): 617
  - metodi di test (@Test): 3228

ATTENZIONE: evitate di giustificare le scelte sul numero di casi di test identificati/progettati/sviluppati con considerazioni tipo: “PER MANTENERE IL NUMERO DI TEST BASSI”, oppure “PER MANTENERE CONTENUTO IL COSTO DELLA FASE DI TEST”

proposta di esercizio preliminare

- considerare il progetto Apache Bookkeeper (o meglio se il progetto assegnatovi secondo l'algoritmo illustrato dal Prof.Falessi)
  - http://bookkeeper.apache.org
  - https://github.com/apache/bookkeeper
- in particolare la documentazione (o la documentazione di riferimento del progetto assegnatovi)
  - http://bookkeeper.apache.org/docs/4.10.0/development/protocol/
- con riferimento alla classe `org.apache.bookkeeper.client.Bookkeeper` (o una classe che studierete nel progetto assegnatovi)
  - https://github.com/apache/bookkeeper/blob/e7430ce25e3609bf77b028e4f0475dad2147c22d/bookkeeper-server/src/main/java/org/apache/bookkeeper/client/BookKeeper.java
- identificare partizioni di dominio per i parametri dei metodi: `createLedger` (o per uno dei metodi della classe di cui sopra)
  - suggerimento, iniziare a considerare: https://github.com/apache/bookkeeper/blob/e7430ce25e3609bf77b028e4f0475dad2147c22d/bookkeeper-server/src/main/java/org/apache/bookkeeper/client/BookKeeper.java#L887
- pianificare l'organizzazione di una descrizione sapendo di dover includere:
  - come sono state identificate le partizioni, e come è stata condotta la boundary analysis
  - proporre un insieme di 10 prompt da sottoporre ad un LLM
    - 4 per zero-shot prompting: ed esempio cambiando le informazioni qualitative del contesto come il livello di dettaglio dei test o di chi li sta chidendo ("professional software tester"), quanto estensiva la ricerca (cohmprensively", numero di test da generare), etc
    - 4 per few-shot prompting: variazioni come nel caso precedente ed in più aggiungere un numero arbitrario di esempi
    - 2 per CoT/ToT: variazioni come nel caso precedente ed in più indicazioni operative su come procedere
  - suggerimenti preliminari di test da includere, con risultato atteso